

Enlace colab

https://colab.research.google.com/drive/1HXKxyv1rnwZTwdg9wNbOHMU-G_U2oMg_?usp=sharing

Capturas originales:



Speed: 295.5ms preprocess, 6.6ms inference, 41.9ms postprocess per image at shape (1, 3, 640, 640)
Results saved to /content/runs/detect/predict

💡 Learn more at <https://docs.ultralytics.com/modes/predict>

1. Detection

YOLOv8 *detection* models have no suffix and are the default YOLOv8 models, i.e. `yolov8`
[Detection Docs](#) for full details.

[3]

✓ 43 s



```
# Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO
```

```
model = YOLO('yolov8n.pt') # load a pretrained YOLOv8n detection model
model.train(data='coco128.yaml', epochs=3) # train the model
model('https://ultralytics.com/images/bus.jpg', save=True) # predict on an image
```

▼

... Ultralytics 8.4.155 🚀 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14GB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=False

WARNING ⚠ Dataset 'coco128.yaml' images not found, missing path '/content/datasets/coco128' -
Downloading <https://ultralytics.com/assets/coco128.zip> to '/content/datasets/coco128.zip'
Unzipping /content/datasets/coco128.zip to /content/datasets/coco128...: 100% -
Dataset download success ✓ (0.9s), saved to /content/datasets

Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf'

	from	n	params	module
0	-1	1	464	ultralytics.nn.modules.conv.Conv
1	-1	1	4672	ultralytics.nn.modules.conv.Conv
2	-1	1	7360	ultralytics.nn.modules.block.C2f
3	-1	1	18560	ultralytics.nn.modules.conv.Conv
4	-1	2	49664	ultralytics.nn.modules.block.C2f
5	-1	1	73984	ultralytics.nn.modules.conv.Conv
6	-1	2	197632	ultralytics.nn.modules.block.C2f
7	-1	1	295424	ultralytics.nn.modules.conv.Conv
8	-1	1	460288	ultralytics.nn.modules.block.C2f
9	-1	1	164608	ultralytics.nn.modules.block.SPPF
10	-1	1	0	torch.nn.modules.upsampling.Upsample
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat

all 128 929 0.7 0.563 0.658 0.487

3 epochs completed in 0.004 hours.

Optimizer stripped from /content/runs/detect/train/weights/last.pt, 6.5MB

Optimizer stripped from /content/runs/detect/train/weights/best.pt, 6.5MB

Validating /content/runs/detect/train/weights/best.pt...

Ultralytics 8.4.155 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
all	128	929	0.696	0.566	0.66	0.488
person	61	254	0.808	0.661	0.763	0.54
bicycle	3	6	0.944	0.333	0.427	0.262
car	12	46	0.748	0.217	0.3	0.19
motorcycle	4	5	0.711	0.8	0.92	0.744
airplane	5	6	0.795	0.833	0.913	0.669
bus	5	7	0.779	0.714	0.715	0.653
train	3	3	0.729	1	0.995	0.763
truck	5	12	1	0.362	0.534	0.33
boat	2	6	0.496	0.167	0.543	0.342
traffic light	4	14	0.684	0.143	0.187	0.135
stop sign	2	2	0.813	1	0.995	0.646
bench	5	9	0.847	0.62	0.722	0.513
bird	2	16	0.926	0.787	0.942	0.641
cat	4	4	0.747	1	0.995	0.808
dog	9	9	0.754	0.667	0.852	0.613
horse	1	2	0.735	1	0.995	0.557
elephant	4	17	0.876	0.882	0.921	0.695
bear	1	1	0.476	1	0.995	0.995
zebra	2	4	0.873	1	0.995	0.957
giraffe	4	9	1	0.861	0.995	0.819
backpack	4	6	0.544	0.211	0.383	0.255
umbrella	4	18	0.713	0.5	0.688	0.461
handbag	9	19	1	0.0883	0.325	0.142
tie	6	7	0.821	0.714	0.76	0.524
suitcase	2	4	0.53	0.575	0.705	0.51
frisbee	5	5	0.563	0.8	0.728	0.652
skis	1	1	0.76	1	0.995	0.464
snowboard	2	7	0.628	0.714	0.713	0.491
sports ball	6	6	0.701	0.402	0.51	0.28
kite	2	10	0.689	0.4	0.572	0.213
baseball bat	4	4	0.44	0.25	0.328	0.213

Zidane

1. Predict

YOLOv8 may be used directly in the Command Line Interface (CLI) with a `yolo` command for a variety of additional arguments, i.e. `imgsz=640`. See a full list of available `yolo` [arguments](#) and other details.



Run inference on an image with YOLOv8n

```
!yolo predict model=yolov8n.pt source='https://ultralytics.com/images/zidane.jpg'
```

... Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to `yolov8n.pt`: 100%
Ultralytics 8.4.155 🚀 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

Downloading <https://ultralytics.com/images/zidane.jpg> to `'zidane.jpg'`: 100%
image 1/1 /content/zidane.jpg: 384x640 2 persons, 1 tie, 6.6ms
Speed: 295.5ms preprocess, 6.6ms inference, 41.9ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /content/runs/detect/predict

💡 Learn more at <https://docs.ultralytics.com/modes/predict>

1. Detection

Bus

```

Downloading https://ultralytics.com/images/bus.jpg to 'bus.jpg': 100% _____
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 6.8ms
Speed: 2.1ms preprocess, 6.8ms inference, 1.5ms postprocess per image at shape (1, 3, 640, 480)
Results saved to /content/runs/detect/predict-2
[ultralytics.engine.results.Results object with attributes:

  boxes: ultralytics.engine.results.Boxes object
  depth: None
  keypoints: None
  masks: None
  names: {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydrant', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'cat', 16: 'dog', 17: 'horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 22: 'zebra', 23: 'giraffe', 24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'suitcase', 28: 'luggage', 29: 'trunk', 30: 'trailer', 31: 'snowboard', 32: 'sports ball', 33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'frisbee', 37: 'hockey stick', 38: 'skis', 39: 'snowboard', 40: 'surfboard', 41: 'kite surfboard', 42: 'pistol', 43: 'rifle', 44: 'shotgun', 45: 'spoon', 46: 'bowl', 47: 'banana', 48: 'apple', 49: 'sandwich', 50: 'orange', 51: 'broccoli', 52: 'potato', 53: 'mushroom', 54: 'egg', 55: 'wine glass', 56: 'cup', 57: 'mug', 58: 'bottle', 59: 'candy', 60: 'dining table', 61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'keyboard', 67: 'cell phone', 68: 'tablet', 69: 'watch', 70: 'clock', 71: 'calendar', 72: 'vase', 73: 'scissors', 74: 'teddy bear', 75: 'hair drier', 76: 'toothbrush'}
  obb: None
  orig_img: array([[119, 146, 172],
                    [121, 148, 174],
                    [122, 152, 177],
                    ...,
                    [161, 171, 188],
                    [160, 170, 187],
                    [160, 170, 187]],
                   [[120, 147, 173],
                    [122, 149, 175],
                    [123, 153, 178],
                    ...,
                    [161, 171, 188],
                    [160, 170, 187],
                    [160, 170, 187]],
                   [[123, 150, 176],
                    [124, 151, 177],
                    [125, 155, 180],
                    ...,
                    [161, 171, 188],
                    [160, 170, 187],
                    [160, 170, 187]]])

```

Mi imagen (Honda civic Type R)



1. Predict

YOLOv8 may be used directly in the Command Line Interface (CLI) with a `yolo` command for a variety of additional arguments, i.e. `imgsz=640`. See a full list of available `yolo` [arguments](#) and other details in

[5]

✓ 6s



```
# Run inference on an image with YOLOv8n
!yolo predict model=yolov8n.pt source='https://mma.prnewswire.com/media/1248063/Honda_Civic_Type_R_Limited_Edition.jpg'
```

▼

```
... Ultralytics 8.4.155 🚀 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs
```

```
Downloading https://mma.prnewswire.com/media/1248063/Honda_Civic_Type_R_Limited_Edition.jpg
image 1/1 /content/Honda_Civic_Type_R_Limited_Edition.jpg: 480x640 1 car, 6.5ms
Speed: 37.1ms preprocess, 6.5ms inference, 17.3ms postprocess per image at shape (1, 3, 480, 640)
Results saved to /content/runs/detect/predict-3
```

```
💡 Learn more at https://docs.ultralytics.com/modes/predict
```

Haz doble clic (o ingresa) para editar



```
# Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO
```

```
model = YOLO('yolov8n.pt') # load a pretrained YOLOv8n detection model
model.train(data='coco128.yaml', epochs=3) # train the model
```

```
model('https://mma.prnewswire.com/media/1248063/Honda_Civic_Type_R_Limited_Edition.jpg?p=twitter')
```

...	tv	2	2	0.556	0.5	0.662	0.579
	laptop	2	3	1	0.432	0.727	0.642
	mouse	2	2	1	0	0.0309	0.00309
	remote	5	8	0.842	0.5	0.59	0.507
	cell phone	5	8	0	0	0.052	0.0328
	microwave	3	3	1	0.611	0.863	0.665
	oven	5	5	0.622	0.4	0.403	0.3
	sink	4	6	0.414	0.167	0.329	0.214
	refrigerator	5	5	0.816	0.6	0.784	0.62
	book	6	29	0.595	0.203	0.428	0.196
	clock	8	9	0.906	0.778	0.906	0.738
	vase	2	2	0.31	1	0.995	0.895
	scissors	1	1	1	0	0.497	0.115
	teddy bear	6	21	0.787	0.528	0.662	0.424
	toothbrush	2	5	1	0.574	0.786	0.482

Speed: 3.2ms preprocess, 2.4ms inference, 0.0ms loss, 3.4ms postprocess per image
Results saved to /content/runs/detect/train-2

Found https://mma.prnewswire.com/media/1248063/Honda_Civic_Type_R_Limited_Edition.jpg locally at

image 1/1 /content/Honda_Civic_Type_R_Limited_Edition.jpg: 480x640 1 car, 6.6ms

Speed: 1.7ms preprocess, 6.6ms inference, 1.6ms postprocess per image at shape (1, 3, 480, 640)

Results saved to /content/runs/detect/predict-4

[ultralytics.engine.results.Results object with attributes:

boxes: ultralytics.engine.results.Boxes object

depth: None

keypoints: None

- ¿Qué clases detectó YOLO en las fotos de Ultralytics y cuáles en la tuya?

Clases detectadas por YOLO en la imagen

Honda_Civic_Type_R_Limited_Edition.jpg:

Tanto la predicción ejecutada desde la línea de comandos como la predicción usando la API de Python detectaron 1 coche en la imagen.

El modelo YOLO, al ser entrenado con coco128.yaml es capaz de detectar un amplio rango de objetos como personas, bicicletas, camiones, semáforos, animales, libros, relojes, etc., como se ve en la lista de names del objeto Results.

- **¿Algún objeto evidente de tu foto no salió etiquetado? ¿Por qué podría pasar (clase que no está en COCO, objeto chico, recorte, umbral de confianza)?**

En la imagen del Honda Civic Type R, el objeto más prominente es el coche, el cual fue correctamente detectado. No parece haber otros objetos **evidentes** de una clase diferente que debieran haber sido detectados por el modelo COCO y que hayan sido omitidos.

- **¿La predicción de la celda CLI y la de model(...) coinciden sobre tu misma imagen?**

Sí, las predicciones coinciden. Ambas aproximaciones detectaron correctamente 1 coche en la imagen del Honda_Civic_Type_R_Limited_Edition.jpg.